TECNOLOGÍAS WEB

Curso 2025/2026

Grado en Ingeniería Informática en Tecnologías de la Información

Doble Grado Telecomunicaciones-Informática

Enunciado del Proyecto - Aplicación Web

Fecha de entrega: 4 de mayo de 2026 a las 23.59h.



Este documento establece los requisitos (mínimos y extras) para la segunda Actividad del proyecto de desarrollo de la asignatura (gestión de pedidos de restaurantes).

Actividad 3: API Rest.

Actividad 3: parte 3 de 3 del proyecto de la aplicación web de la asignatura Tecnologías Web.

En esta actividad los estudiantes deberán desarrollar el backend de una plataforma de gestión de restaurantes y pedidos mediante una **API REST**. La API deberá permitir la gestión de usuarios, restaurantes, platos, pedidos y valoraciones, exponiendo los recursos a través de **URIs**, utilizando correctamente los **métodos HTTP** y devolviendo la información en formato **JSON**.

La actividad se centra en la capa de servicio. Por tanto, no se evaluará una interfaz JSP/HTML renderizada en servidor, sino el diseño, implementación y documentación de los servicios REST, así como su correcto funcionamiento.

REQUISITOS MÍNIMOS

En caso de que se cumplan todos los requisitos mínimos (y se implementen correctamente), la calificación será de 6. Esta calificación se puede mejorar con funcionalidades adicionales.

Sin embargo, en caso de que no se cumpla alguno de los requisitos mínimos, la calificación será de 3 sin posibilidad de obtener una mejor calificación implementando funcionalidades adicionales.

Los requisitos mínimos se dividen en funcionales y no funcionales.

Requisitos no funcionales:

1. **Arquitectura REST:** La solución deberá seguir una arquitectura orientada a recursos. Cada recurso deberá estar identificado mediante una URI clara y coherente.
2. **Formato de intercambio:** La API consumirá y producirá información en formato **JSON**.
3. **Métodos HTTP y códigos de estado:** La API deberá utilizar correctamente al menos los métodos **GET, POST, PUT y DELETE**, así como devolver códigos de estado HTTP adecuados, por ejemplo `200`, `201`, `204`, `400`, `401`, `403`, `404` o `409`, según corresponda.
4. **Seguridad:** La API deberá garantizar que un usuario autenticado no pueda acceder, modificar o eliminar recursos de otros usuarios salvo que sean públicos o compartidos expresamente. La comunicación deberá realizarse de forma segura.

5. **Buenas prácticas:** El código deberá seguir buenas prácticas de organización, separación por capas y reutilización, conforme a lo trabajado en la asignatura.
6. **Documentación de la API:** Cada endpoint deberá estar documentado indicando URI, método HTTP, parámetros, cuerpo de petición, respuesta y códigos de estado posibles.

Requisitos funcionales:

1. **Registro y acceso de usuarios**: Cualquier persona puede registrarse como usuario de la aplicación web a través de un correo electrónico y una contraseña robusta (que al menos contendrá 8 dígitos entre los que se incluye un número, una mayúscula, una minúscula y un carácter alfanumérico). Adicionalmente se podrán añadir otros datos como nombre, apellidos y otros datos que se consideren necesarios. Tras realizar el alta, cada usuario podrá editar su información y eliminar su cuenta en cualquier momento. Para cumplir satisfactoriamente con este requisito el usuario debe ver la información previamente cargada de su perfil cuando lo está editando. Ejemplo de endpoints pueden ser:
   - `POST /api/usuarios`
   - `GET /api/usuarios/{id}`
   - `PUT /api/usuarios/{id}`
   - `DELETE /api/usuarios/{id}`
   - `POST /api/sesiones -> Requisito Extra (+0,5 puntos.)`

2. **Gestión de restaurantes**: Cualquier usuario puede crear, ver, editar y eliminar un restaurante que contenga, al menos (se deben respetar los tipos de campos):
   - Nombre: Text
   - Dirección: Textarea
   - Teléfono: Tel
   - Correo de contacto: Email
   - Rango de precio: Numbers
   - Categorías: Checkbox
   - Media de valoraciones: a elección por parte de los estudiantes
   - Bike Friendly: Sí/No (Radio button)

   Se pueden añadir otros campos que se consideren necesarios.

   Para cumplir satisfactoriamente con este requisito cuando el usuario propietario del restaurante **solo podrá editar y eliminar aquellos restaurantes de los que sea propietario** y, además, en el momento de editar la información debe aparecer previamente cargada y almacenada en el sistema cuando el restaurante se introdujo.

   La API deberá permitir a un usuario autenticado crear, consultar, actualizar y eliminar restaurantes y deberá impedir que un usuario modifique o elimine restaurantes que no sean de su propiedad.

Ejemplo de endpoints pueden ser:

- `GET /api/restaurantes/all -> Listado de restaurantes.`
- `GET /api/restaurantes -> Restaurantes del usuario.`
- `GET /api/restaurantes/{id}`
- `POST /api/restaurantes`
- `PUT /api/restaurantes/{id}`
- `DELETE /api/restaurantes/{id}`

3. **Gestión de platos de un restaurante**: La API deberá permitir al propietario de un restaurante gestionar los platos asociados a dicho restaurante. Por cada plato, al menos, se dispondrá de la siguiente información:
   - Nombre del plato.
   - Descripción.
   - Precio.

   Ejemplo de endpoints podrían ser:
   - `GET /api/restaurantes/{id}/platos`
   - `POST /api/restaurantes/{id}/platos`
   - `PUT /api/platos/{id}`
   - `DELETE /api/platos/{id}`

4. **Gestión de pedidos:** La API deberá permitir a cualquier usuario autenticado realizar pedidos de platos de un restaurante y consultar su histórico de pedidos. Cabe destacar que un mismo usuario podrá añadir restaurantes y platos para sus restaurantes, así como realizar pedidos de cualquier restaurante.

   Ejemplo de endpoints podrían ser:

   - `POST /api/pedidos`
   - `GET /api/usuarios/{id}/pedidos`
   - `GET /api/pedidos/{id}`

5. **Valoración**: La API deberá permitir que un usuario valore un restaurante con una puntuación de 1 a 5 y un comentario opcional. Un usuario no podrá realizar más de una valoración por restaurante. También deberá permitir consultar restaurantes ordenados por valoración media.

   Ejemplo de endpoints podrían ser:

   - `POST /api/restaurantes/{id}/valoraciones`
   - `GET /api/restaurantes/{id}/valoraciones`
   - `GET /api/restaurantes?sort=valoracionMedia`

6. **Buscar restaurante**: un usuario puede buscar un restaurante concreto por su localidad o escribiendo las palabras relativas al título o su descripción. La API deberá permitir buscar restaurantes por localidad o por términos presentes en su nombre o descripción.

   Ejemplo de endpoints podrían ser:

   - `GET /api/restaurantes?localidad=Mérida`
   - `GET /api/restaurantes?q=pizza`

7. **Estado del restaurante**: un usuario puede cambiar el estado de sus restaurantes de “acepta pedidos” a “no acepta pedidos” para indicar que un restaurante concreto no permite envíos en un momento determinado y no se encuentra disponible. La aplicación debe permitir a los usuarios filtrar los restaurantes para mostrar solo aquellos que aceptan pedidos (es decir, ocultar los restaurantes que no aceptan pedidos). De este modo la aplicación contará con tres apartados:
   - **Todos:** donde aparecen todos los restaurantes acepten pedidos o no.
   - **Aceptan pedidos:** solo los restaurantes que aceptan pedidos.
   - **No aceptan pedidos:** solo los restaurantes que no aceptan pedidos.

   Ejemplo de endpoints podrían ser:

   - `PUT /api/restaurantes/{id}/estado`
   - `GET /api/restaurantes?estado=acepta`
   - `GET /api/restaurantes?estado=no-acepta`

REQUISITOS EXTRAS

Al implementar los requisitos adicionales, los estudiantes pueden obtener los 4 puntos adicionales para alcanzar la calificación máxima de 10, siempre y cuando en los requisitos mínimos se obtenga al menos la calificación de 5.

Cada extra puede agregar cierta cantidad (ver cada uno para más detalles) a la calificación final del proyecto.

1. **Gestión de categorías (hasta 1 punto):** la aplicación web permite la creación, edición, listado y eliminación de las categorías para clasificar los restaurantes de manera adecuada:
   - Gestión de Categorías (operaciones CRUD).
   - Agrega una o más categorías a un restaurante.
   - Elimina una o más categorías de un restaurante.
   - Acceso rápido a los restaurantes con la misma categoría (por ejemplo, ofreciendo un menú de categorías en alguna parte de la

página)

2. **Restaurantes favoritos (hasta 0,75 puntos):** los usuarios podrán tener un conjunto de restaurantes favoritos. En este caso, en su perfil aparecerá el listado de restaurantes favoritos. Un mismo usuario no puede añadir dos veces el mismo restaurante a su lista de favoritos. Sí podrá quitar un restaurante de sus favoritos. En el listado de favoritos debe aparecer al menos el nombre del restaurante y la imagen que lo identifique (requiere implementar el requisito extra de imágenes para restaurantes).

3. **Búsqueda avanzada (hasta 1 punto):** cualquier usuario puede buscar restaurantes escribiendo las palabras que está buscando y luego filtrar sus resultados de búsqueda por:
   - Categorías (hasta 0,2)
   - Localidad (hasta 0,2)
   - Comentarios (hasta 0,2)
   - Platos que oferte (hasta 0,2)
   - Rango de precios (hasta 0,2)

4. **Restaurantes relacionados (hasta 0,80 puntos)**: cuando el usuario está viendo los detalles de un restaurante, se presenta una lista de restaurantes relacionados en función de la categoría, localidad, precio o valoración.

5. **Gestión de imágenes de usuarios y restaurantes (CRUD) (hasta 0,50 puntos):** se permite añadir, editar y eliminar imágenes identificativas propias tanto para los usuarios como para los restaurantes y platos almacenados en el sistema.

6. **Gestión de mensajería interna (hasta 1,25 puntos):** El sistema permite el intercambio privado e interno de mensajes entre usuarios que solo podrán ver de manera individual en su perfil.

7. **Gestión de repartidores (hasta 1,5 puntos):** el sistema permite que usuarios con el rol de repartidor puedan asignarse pedidos que quieren repartir. Deben tener acceso al listado completo de pedidos realizados por usuario y poder asignarse un pedido para repartirlo. Quedará registrado el listado de pedidos repartidos por cada repartidor.

8. **Acceso rápido a promociones (hasta 0,5 puntos):** el sistema a través de imágenes identificativas proporciona un acceso directo a las categorías para promocionar restaurantes de manera rápida. El dueño del restaurante podrá añadir promociones indicando el nombre del plato, su imagen y el porcentaje de descuento.

9. **Panel de redes sociales (hasta 0,5 puntos):** en el detalle del restaurante aparece un panel con el perfil en redes sociales del restaurante (e.g. los tweets). Dado que cada restaurante tendrá su propio perfil, éste podrá ser especificado en la creación y edición del mismo.

10. **Estadísticas gráficas (hasta 2 puntos)**:
    - El sistema proporciona a cada usuario un apartado privado donde se muestren a través de gráficas las estadísticas relativas a la actividad de sus restaurantes.
    - Las estadísticas se mostrarán en función de la temporalidad: día, semana, mes y año.
    - Los datos que se deben almacenar y mostrar en las estadísticas son:
        - Número de restaurantes disponibles por un usuario.
        - Número de pedidos recibidos por los restaurantes de un usuario.
        - La valoración media que reciben los restaurantes de un usuario.
        - El número de pedidos realizados por un usuario.

11. **Cálculo de la cuenta instantánea (hasta 0,75 puntos):** mientras el usuario añade platos al pedido que va realizando el cálculo de la cuenta se debe ir actualizando de manera instantánea según añada platos (suma) y según quite platos (resta).

12. **Interfaz de usuario de alta calidad (hasta 2 puntos).** Se valorará adicionalmente el cuidado y la calidad de la interfaz de usuario, así como que sea “responsive”.

13. **Cualquier otra funcionalidad que el estudiante añada y pueda justificar como valor significativo y diferenciador al proyecto.** (Hasta 1 punto según su complejidad).

SITUACIONES QUE SE DEBEN EVITAR

Algunas de las situaciones pueden penalizar la calificación final hasta 1 punto cada una son las siguientes:

- Endpoints incoherentes o mal diseñados.
- Uso incorrecto de métodos HTTP.
- Respuestas JSON inválidas.
- Ausencia de control de errores.
- Fuga de información sensible.
- Acceso no autorizado a recursos de otros usuarios;
- Funcionalidades aparentemente disponibles, pero no operativas.

ALGUNAS CONSIDERACIONES IMPORTANTES

Fecha de entrega: 4 de mayo de 2026 a las 23.59h.

Los estudiantes deben entregar:

- Un archivo **.zip** con **Nombre_Apellido1_Apellido2** que contenga el fichero .zip que Eclipse genera al utilizar la opción Exportar (botón derecho del ratón sobre vuestro proyecto en Eclipse). Este fichero puede ser importado fácilmente a otra instalación de Eclipse.
- **La base de datos** utilizada en el proyecto y el nombre de este archivo debe estar compuesto por las iniciales de nombre y apellidos (por ejemplo, ABS para Armando Bronca Segura). En el código del proyecto, la base de datos deberá estar referenciada con ese mismo nombre (en el ejemplo, ABS.db).
- **Fichero .json** exportado desde la herramienta utilizada para probar la API, por ejemplo, API Tester, Postman, Thunder Client o equivalente. Esta colección deberá:
  - Incluir las peticiones necesarias para comprobar el funcionamiento de los endpoints implementados.
  - Estar organizada de forma clara.
  - Permitir reproducir las pruebas.
  - Contener ejemplos representativos de consulta, alta, edición y borrado cuando proceda.
  - Reflejar, cuando sea posible, los códigos de respuesta esperados.

- **Documento PDF** que contenga el **nombre del usuario** con el que se ejecuta la prueba y un conjunto de evidencias (capturas) de la ejecución y tester de los endpoints del proyecto.